# UNIVERSIDAD VERACRUZANA — FACULTAD DE INSTRUMENTACIÓN ELECTRÓNICA

## Reporte de Práctica de Laboratorio — Programación Orientada a Objetos

### EE: Temas Selectos de IIE II (Código: IEDI 18005)

#### Facultad de Instrumentación Electrónica — Universidad Veracruzana

---

### Información General del Equipo

- **Número de Práctica:** Práctica 2
- **Título de la Práctica:** Sistema Modular de Nodos IoT (Herencia y Polimorfismo en Python)
- **Fecha de Entrega:** 07/11/2026
- **Enlace al Repositorio de GitHub:** https://github.com/AndreVg4/POO_Equipo5.git
- **Ruta de Entrega en Repositorio:** `practica-02/docs/Reporte_Evidencias_P2.pdf`

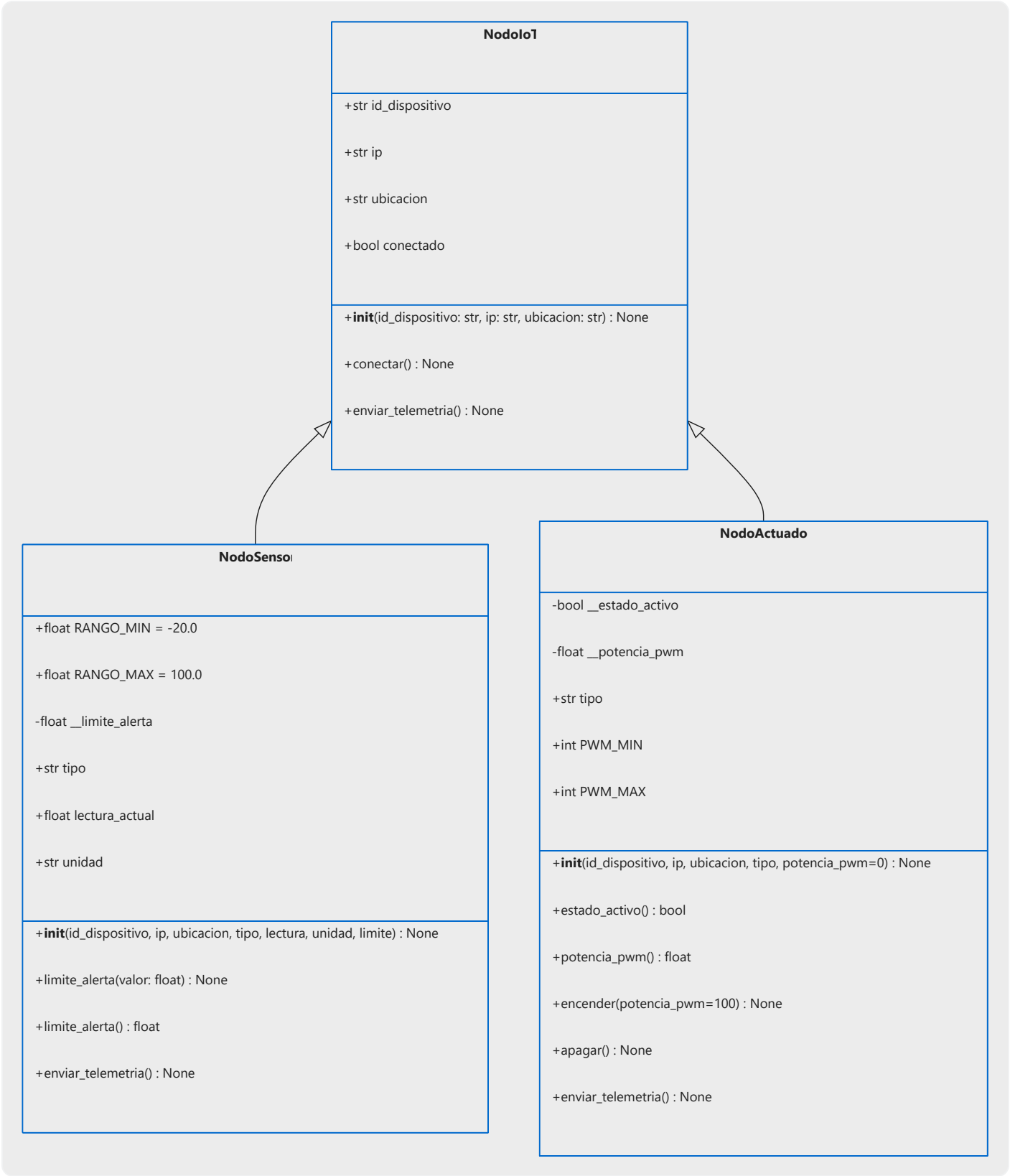
| Nombre Completo del Integrante | Matrícula UV | Usuario de GitHub |
|---|---|---|
| Andre Vega Morales | S22013537 | @AndreVg4 |
| Ricardo Abraham Sosa Calvario | S22013518 | @RicardoS22 |
| Isaac Vazquez Carrizo | S22013542 | @IsaacVzC |
| Eduardo Ramos Mateos | S22013532 | @Eduardo-Ramoss |

---

### Sección 1: Abstracción de Hardware a POO (Mapeo de Clases)

#### 1.1. Diagrama de Clases UML

*El siguiente diagrama modela la jerarquía de herencia hacia la superclase `NodoIoT`, especificando visibilidades (+ público, - privado), tipos de datos y parámetros:*




### 1.2. Mapeo de Atributos y Especificaciones Técnicas

*(Respuesta a: ¿Cómo se representan las especificaciones técnicas del sensor/nodo físico como atributos en tu clase?)*

- **Superclase `NodoIoT` (Atributos centralizados de red de instrumentación):**

    - `id_dispositivo`: Representa el identificador único o etiqueta física del instrumento (*Tag ID* o dirección física/MAC de la interfaz de comunicaciones).
    - `ip`: Dirección de red IPv4 asignada a la tarjeta lógica para su enlace dentro de la malla local IoT.
    - `ubicacion`: Ubicación física o celda de instalación industrial del nodo (ej. "*Invernadero*", "*Laboratorio*", "*Cisterna*").
    - `conectado`: Bandera booleana de estado de capa de transporte que indica si el canal de enlace está establecido hacia el maestro SCADA.

- **Subclase `NodoSensor` (Variables físicas de sensado):**
    - `RANGO_MIN` / `RANGO_MAX`: Constantes de clase que modelan los límites nominales de acondicionamiento físico del transductor ($-20.0^\circ\text{C}$ a $100.0^\circ\text{C}$).
    - `tipo`: Parámetro físico bajo monitoreo (*Temperatura*, *Luminosidad*).
    - `lectura_actual`: Variable procesada en unidades de ingeniería proveniente de la transducción y etapa ADC.
    - `unidad`: Unidad física del sistema internacional correspondiente ($^\circ\text{C}$, $\%$, etc.).
    - `__limite_alerta`: Umbral de advertencia crítica superior para disparo seguro de alarmas de proceso.

- **Subclase `NodoActuador` (Variables de potencia y control):**
    - `PWM_MIN` / `PWM_MAX`: Constantes enteras ($0$ a $100$) que acotan el rango de modulación física admisible.
    - `tipo`: Elemento final de control instrumentado (*Ventilador*, *Bomba de Agua*).
    - `__estado_activo`: Bandera discreta de conducción/bloqueo de la etapa de potencia o relé.
    - `__potencia_pwm`: Ciclo de trabajo (*Duty Cycle*) de la modulación PWM aplicado a la carga ($0.0\%$ a $100.0\%$).

### 1.3. Mapeo de Métodos (Acciones Físicas ➔ Métodos Públicos)

*(Respuesta a: ¿Qué acciones físicas del hardware se mapearon como métodos públicos en tu objeto?)*

- `conectar()` en `NodoIoT`: Abre lógicamente el socket de red del dispositivo con el concentrador IoT/SCADA y actualiza `conectado = True`.
- `enviar_telemetria()`:
    - En **`NodoSensor`**: Lee el valor del transductor, emite la trama de instrumentación con su unidad y ejecuta una rutina de supervisión de límites que emite una alarma si

$\text{lectura_actual} > \text{__limite_alerta}$.
    - En `NodoActuador`: Consulta y reporta el estado de funcionamiento (encendido/apagado) junto con el ciclo de trabajo PWM activo.
- `encender(potencia_pwm=100)` en `NodoActuador`: Energiza el actuador aplicando el porcentaje PWM indicado tras validar el rango.
- `apagar()` en `NodoActuador`: Desenergiza la etapa de conmutación de potencia y fuerza el PWM a $0.0\%$.

#### 1.4. Encapsulamiento y Protección de Datos Crudos

*(Respuesta a: ¿Cómo aplicaste el encapsulamiento para proteger los datos crudos del sensor antes de ser procesados?)*

Se protegió el acceso a las variables críticas declarándolas privadas mediante el prefijo de doble guion bajo (`__`):

- En `NodoSensor`, el atributo `__limite_alerta` está estrictamente encapsulado. Se implementaron decoradores `@property` y `@limite_alerta.setter` que validan entradas usando `isinstance(valor, numbers.Real) and not isinstance(valor, bool)` para rechazar booleanos y cadenas con `TypeError`, e imponen el intervalo térmico válido $[-20.0, 100.0]^{\circ}\text{C}$ levantando `ValueError`.
- En `NodoActuador`, las variables `__estado_activo` y `__potencia_pwm` se encapsulan para impedir que un comando externo fije valores que excedan el rango $[0, 100]\%$, protegiendo al actuador frente a comandos inválidos.

## Sección 2: Bitácora de Depuración y Errores (The "Bug" Log)

| # | Falla Física o Lógica Detectada | Causa Raíz (Hardware / Software) | Solución Técnica Implementada en Código / Conexión |
|---|---|---|---|
| **1** | Asignación accidental de valores booleanos o tipos incompatibles en el umbral `limite_alerta` del sensor (ej. un valor `True` era procesado como `1` numérico | En la jerarquía de tipos de Python, `bool` es subclase directa de `int`, por lo que una validación simple con `isinstance(valor, (int, float))` evalúa como válida una bandera booleana. | Se estructuró una regla estricta en el *setter* utilizando `isinstance(valor, numbers.Real) and not isinstance(valor, bool)` para rechazar booleanos y strings arrojando `TypeError`, verificando además que el valor pertenezca al rango $[-20.0, 100.0]^{\circ}\text{C}$ con `ValueError`. |

| # | Falla Física o Lógica Detectada | Causa Raíz (Hardware / Software) | Solución Técnica Implementada en Código / Conexión |
|---|---|---|---|
| | sin levantar excepción). | | |
| **2** | Ingreso de valores de ciclo de trabajo PWM fuera del intervalo admisible ($< 0\%$ o $> 100\%$), lo que provocaría sobresaturación en el control del actuador. | Ausencia de validación de umbrales máximos/mínimos al invocar `encender(potencia)` desde la interfaz del usuario. | Se añadieron guardas de validación en el *setter* de `__potencia_pwm` y en `encender()`, lanzando `ValueError` si la entrada sale del intervalo $[0, 100]$, garantizando que el actuador opere exclusivamente dentro del rango de trabajo seguro. |
| **3** | La impresión secuencial de telemetrías llenaba indefinidamente el terminal, dificultando la visualización de alertas en tiempo real. | Uso de `print()` directo no estructurado en cada ciclo de telemetría sin refresco de interfaz ni buffer circular de eventos. | Se construyó una interfaz HMI limpia con borrado de pantalla multiplataforma (`os.system('cls' if os.name == 'nt' else 'clear')`) y un buffer SCADA acotado con `collections.deque(maxlen=5)` para retener los últimos eventos de red sin perder visibilidad. |

---

## Sección 3: Justificación del Diseño Orientado a Objetos

1. **Herencia y Reutilización de Código:**
    - **Superclase:** `NodoIoT`.
    - **Subclases:** `NodoSensor` y `NodoActuador`.
    - **Atributos heredados:** `id_dispositivo`, `ip`, `ubicacion` y `conectado`.
    - **Eficiencia en diseño electrónico y modular:** En una arquitectura IoT de instrumentación, todos los dispositivos de campo comparten de manera obligatoria la pila de comunicaciones (dirección de red, identificador de bus y estado del enlace). Si se programaran de forma plana e independiente (como en la Práctica 1), se tendría código duplicado para el manejo de sockets y metadatos de red en cada transductor. Utilizar la superclase `NodoIoT` e invocar `super().__init__(id_dispositivo, ip, ubicacion)` permite centralizar la infraestructura de red en un único módulo base, garantizando

mantenibilidad y permitiendo añadir nuevos sensores o actuadores simplemente extendiendo la clase base.

2. **Polimorfismo (Bucle sin condicionales):**
    - **Implementación:** Ambas subclases sobrescriben el método `enviar_telemetria()`.
    - **Demostración en bucle polimórfico:**

```
red_iot = [sensor_temp, actuador_ventilador, sensor_luz, actuador_bomba]

# Bucle polimórfico: no requiere if/else ni verificación de tipo
for dispositivo in red_iot:
    dispositivo.enviar_telemetria()
```

    - **Justificación de diseño:** La estación maestra de monitoreo no necesita conocer la clase interna ni recurrir a condicionales (`if isinstance(...)`) para consultar a los nodos. En tiempo de ejecución, el despachador dinámico ejecuta la versión correspondiente: el sensor formatea mediciones y valida límites de alerta, mientras que el actuador informa su estado de relé y ciclo de trabajo PWM.

3. **Encapsulamiento y Protección de Datos:**
    - **Atributos protegidos:** `__limite_alerta` en `NodoSensor`, `__estado_activo` y `__potencia_pwm` en `NodoActuador`.
    - **Justificación de diseño:** Evita que scripts externos o ingresos erróneos de usuario corrompan las variables críticas del sistema. El uso de `@property` actúa como una capa de acondicionamiento de señal digital que valida tipos y rangos físicos antes de permitir cualquier escritura en la memoria del objeto.

## Sección 4: Evidencias de Funcionamiento (Simulación y Software)

### 4.1. Captura de Pantalla (Evidencia de Software y Consola HMI)

*Salida estructurada en la terminal de Python mostrando la creación de instancias, la invocación de `super().__init__()`, el barrido polimórfico de telemetría de `red_iot` y el bloqueo de excepciones por validación en los setters:*

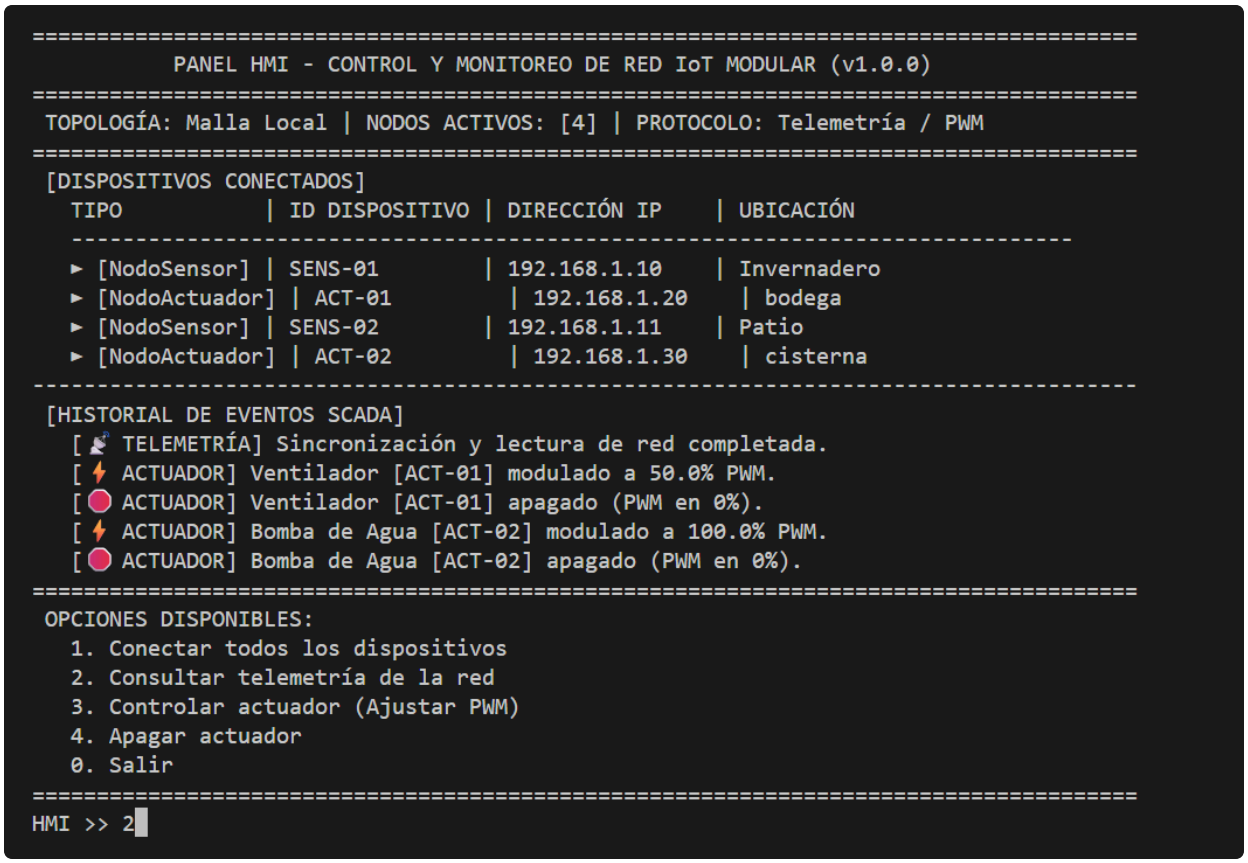



*Figura 1: Panel HMI principal mostrando topología, nodos activos y buffer circular SCADA.*

```
================================== TELEMETRIA ==================================

--- Telemetría NodoSensor ---
ID: SENS-01 | IP: 192.168.1.10 | Ubicación: Invernadero (Conectado)
Sensor: Temperatura | Lectura: 45.5 °C
Límite configurado: 40.0 °C -> Estado: ¡ALERTA SUPERADA!
-----------------------------


--- Telemetría NodoActuador ---
ID: ACT-01 | IP: 192.168.1.20 | Ubicación: bodega (Conectado)
Tipo de actuador: Ventilador
Estado del actuador: Apagado
Potencia PWM: 0.0%
-------------------------------


--- Telemetría NodoSensor ---
ID: SENS-02 | IP: 192.168.1.11 | Ubicación: Patio (Conectado)
Sensor: Luminosidad | Lectura: 65.0 %
Límite configurado: 80.0 % -> Estado: Lectura estable
-----------------------------


--- Telemetría NodoActuador ---
ID: ACT-02 | IP: 192.168.1.30 | Ubicación: cisterna (Conectado)
Tipo de actuador: Bomba de Agua
Estado del actuador: Apagado
Potencia PWM: 0.0%
-------------------------------

================================================================================

Presiona Enter para regresar a la HMI...
```

*Figura 2: Ejecución del barrido polimórfico de telemetría reportando mediciones, alertas y estados de actuadores.*

---

### 4.2. Enlace Directo al Video Demostrativo (Máximo 3 minutos)

- **Enlace de Visualización:** https://drive.google.com/drive/folders/1n2wlGuKS5DbuUAE0eu9uoGqaqvCqjjQl?usp=sharing

*(El video demuestra la ejecución de la simulación en consola, el funcionamiento del menú interactivo, la selección y modulación PWM de los actuadores y el comportamiento polimórfico de la telemetría ante las alarmas).*